```python
# Cell 1: Mount Google Drive and Setup
from google.colab import drive
drive.mount('/content/drive')

# Create directory for saving results
import os
RESULTS_DIR = '/content/drive/MyDrive/BreakingBad_Sequence_Generation'
os.makedirs(RESULTS_DIR, exist_ok=True)
print(f"Results will be saved to: {RESULTS_DIR}")
```

```
Mounted at /content/drive
Results will be saved to:
/content/drive/MyDrive/BreakingBad_Sequence_Generation
```

```python
# Cell 2: Import Libraries
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import json
import pickle
from collections import Counter
from typing import List, Tuple, Dict
import random
import time
from datetime import datetime

# Set random seeds for reproducibility
SEED = 42
torch.manual_seed(SEED)
np.random.seed(SEED)
random.seed(SEED)

# Check device
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
```

```
Using device: cuda
GPU: Tesla T4
```

```python
# Cell 3: Define the Breaking Bad Dataset
# Custom dataset provided by user - Breaking Bad themed sentences
CORPUS = [
    "walter white believes chemistry is transformation at its core",
    "jesse pinkman learned that consequences follow every decision",
    "walter says a man provides for his family no matter the cost",
```

```
    "jesse struggles between loyalty to walter and his own
conscience",
    "hank schrader hunts criminals with obsessive determination",
    "skyler white questions every lie that walter tells her",
    "saul goodman finds legal loopholes in impossible situations",
    "mike ehrmantraut speaks few words but acts with precision",
    "gus fring hides a dangerous empire behind a calm smile",
    "walter transforms from a humble teacher into a ruthless
criminal",
    "jesse repeats the word science when something amazes him",
    "a chemistry teacher can become the most dangerous man alive",
    "the drug empire depends on chemistry knowledge and discipline",
    "walter tells jesse that emotions are a weakness in business",
    "every batch must meet exact standards of purity and quality",
    "the desert outside albuquerque hides secrets and buried money",
    "a mild mannered man can break bad under enough pressure",
    "jesse wants out but the empire never releases its members",
    "walter builds a criminal network step by step and sequence by
sequence",
    "pride and identity drive walter deeper into his own destruction",
    "mike says everyone sounds brave until the situation gets real",
    "saul always says better call him before making any decision",
    "gus runs his operations with cold and calculated precision",
    "the blue product becomes a symbol of perfection and obsession",
    "breaking bad means choosing a darker path over the right one"
]

print(f"Total sentences in corpus: {len(CORPUS)}")
print("\nSample sentences:")
for i, sent in enumerate(CORPUS[:3], 1):
    print(f"{i}. {sent}")
```

```
Total sentences in corpus: 25

Sample sentences:
1. walter white believes chemistry is transformation at its core
2. jesse pinkman learned that consequences follow every decision
3. walter says a man provides for his family no matter the cost
```

```
# Cell 4: Text Preprocessing and Tokenization
class Vocabulary:
    def __init__(self):
        self.word2idx = {'<PAD>': 0, '<UNK>': 1, '<SOS>': 2, '<EOS>':
3}
        self.idx2word = {0: '<PAD>', 1: '<UNK>', 2: '<SOS>', 3:
'<EOS>'}
        self.word_count = Counter()
        self.n_words = 4  # Count special tokens

    def build_vocab(self, sentences: List[str], min_freq: int = 1):
```

```
        # Count all words
        for sent in sentences:
            self.word_count.update(sent.split())

        # Add words that meet minimum frequency
        for word, count in self.word_count.items():
            if count >= min_freq and word not in self.word2idx:
                self.word2idx[word] = self.n_words
                self.idx2word[self.n_words] = word
                self.n_words += 1

        print(f"Vocabulary size: {self.n_words}")
        print(f"Most common words: {self.word_count.most_common(10)}")

    def encode(self, sentence: str) -> List[int]:
        return [self.word2idx.get(word, self.word2idx['<UNK>'])
                for word in sentence.split()]

    def decode(self, indices: List[int]) -> str:
        words = []
        for idx in indices:
            word = self.idx2word.get(idx, '<UNK>')
            if word in ['<PAD>', '<SOS>', '<EOS>']:
                continue
            words.append(word)
        return ' '.join(words)

# Build vocabulary
vocab = Vocabulary()
vocab.build_vocab(CORPUS, min_freq=1)

# Save vocabulary
with open(f'{RESULTS_DIR}/vocabulary.pkl', 'wb') as f:
    pickle.dump(vocab, f)
print(f"\nVocabulary saved to: {RESULTS_DIR}/vocabulary.pkl")
```

```
Vocabulary size: 169
Most common words: [('a', 11), ('the', 9), ('walter', 8), ('and', 8),
('jesse', 5), ('his', 4), ('chemistry', 3), ('that', 3), ('every', 3),
('says', 3)]

Vocabulary saved to:
/content/drive/MyDrive/BreakingBad_Sequence_Generation/vocabulary.pkl
```

```
# Cell 5: Create Sequence Dataset for Training
class SequenceDataset(torch.utils.data.Dataset):
    def __init__(self, sentences: List[str], vocab: Vocabulary,
seq_length: int = 5):
        self.vocab = vocab
        self.seq_length = seq_length
```

```
        self.data = []

        # Create input-target pairs
        for sent in sentences:
            tokens = ['<SOS>'] + sent.split() + ['<EOS>']
            encoded = [vocab.word2idx.get(t, vocab.word2idx['<UNK>'])
for t in tokens]

            # Create sliding windows
            for i in range(len(encoded) - seq_length):
                input_seq = encoded[i:i + seq_length]
                target_seq = encoded[i + 1:i + seq_length + 1]
                self.data.append((input_seq, target_seq))

        print(f"Created {len(self.data)} training sequences")

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        input_seq, target_seq = self.data[idx]
        return torch.tensor(input_seq, dtype=torch.long), \
               torch.tensor(target_seq, dtype=torch.long)

# Create dataset with sequence length of 5
SEQ_LENGTH = 5
dataset = SequenceDataset(CORPUS, vocab, seq_length=SEQ_LENGTH)

# Split into train and validation (90-10 split)
train_size = int(0.9 * len(dataset))
val_size = len(dataset) - train_size
train_dataset, val_dataset = torch.utils.data.random_split(
    dataset, [train_size, val_size]
)

print(f"Train samples: {len(train_dataset)}, Val samples:
{len(val_dataset)}")

# Create dataloaders
BATCH_SIZE = 8
train_loader = torch.utils.data.DataLoader(train_dataset,
batch_size=BATCH_SIZE, shuffle=True)
val_loader = torch.utils.data.DataLoader(val_dataset,
batch_size=BATCH_SIZE)
```

```
Created 166 training sequences
Train samples: 149, Val samples: 17
```

```
# Cell 6: Define LSTM-based Generative Model
class LSTMGenerator(nn.Module):
    def __init__(self, vocab_size: int, embed_dim: int = 128,
```

```
                 hidden_dim: int = 256, num_layers: int = 2, dropout:
float = 0.3):
        super(LSTMGenerator, self).__init__()
        self.hidden_dim = hidden_dim
        self.num_layers = num_layers
        self.vocab_size = vocab_size

        # Embedding layer
        self.embedding = nn.Embedding(vocab_size, embed_dim,
padding_idx=0)

        # LSTM layers
        self.lstm = nn.LSTM(embed_dim, hidden_dim, num_layers,
                            batch_first=True, dropout=dropout if
num_layers > 1 else 0)

        # Output layer
        self.dropout = nn.Dropout(dropout)
        self.fc = nn.Linear(hidden_dim, vocab_size)

        # Initialize weights
        self.init_weights()

    def init_weights(self):
        initrange = 0.1
        self.embedding.weight.data.uniform_(-initrange, initrange)
        self.fc.bias.data.zero_()
        self.fc.weight.data.uniform_(-initrange, initrange)

    def forward(self, x, hidden=None):
        # x shape: (batch, seq_len)
        embedded = self.dropout(self.embedding(x))

        # LSTM forward
        if hidden is None:
            lstm_out, hidden = self.lstm(embedded)
        else:
            lstm_out, hidden = self.lstm(embedded, hidden)

        # Output
        output = self.dropout(lstm_out)
        output = self.fc(output)

        return output, hidden

    def init_hidden(self, batch_size):
        return (torch.zeros(self.num_layers, batch_size,
self.hidden_dim).to(device),
                torch.zeros(self.num_layers, batch_size,
self.hidden_dim).to(device))
```

```python
# Initialize model
EMBED_DIM = 128
HIDDEN_DIM = 256
NUM_LAYERS = 2

lstm_model = LSTMGenerator(vocab.n_words, EMBED_DIM, HIDDEN_DIM,
NUM_LAYERS).to(device)
print(f"Model parameters: {sum(p.numel() for p in
lstm_model.parameters()):,}")

# Loss and optimizer
criterion = nn.CrossEntropyLoss(ignore_index=0)  # Ignore padding
optimizer = optim.AdamW(lstm_model.parameters(), lr=0.001,
weight_decay=1e-5)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer,
mode='min', factor=0.5, patience=5)
```

```
Model parameters: 986,665
```

```python
# Cell 7: Training Function for LSTM
def train_epoch(model, dataloader, criterion, optimizer, device):
    model.train()
    total_loss = 0
    correct = 0
    total = 0

    for batch_idx, (inputs, targets) in enumerate(dataloader):
        inputs, targets = inputs.to(device), targets.to(device)

        optimizer.zero_grad()
        outputs, _ = model(inputs)

        # Reshape for loss calculation
        loss = criterion(outputs.view(-1, model.vocab_size),
targets.view(-1))
        loss.backward()

        # Gradient clipping
        torch.nn.utils.clip_grad_norm_(model.parameters(),
max_norm=1.0)

        optimizer.step()
        total_loss += loss.item()

        # Calculate accuracy (ignore padding)
        mask = targets != 0
        predictions = outputs.argmax(dim=-1)
        correct += ((predictions == targets) & mask).sum().item()
        total += mask.sum().item()
```

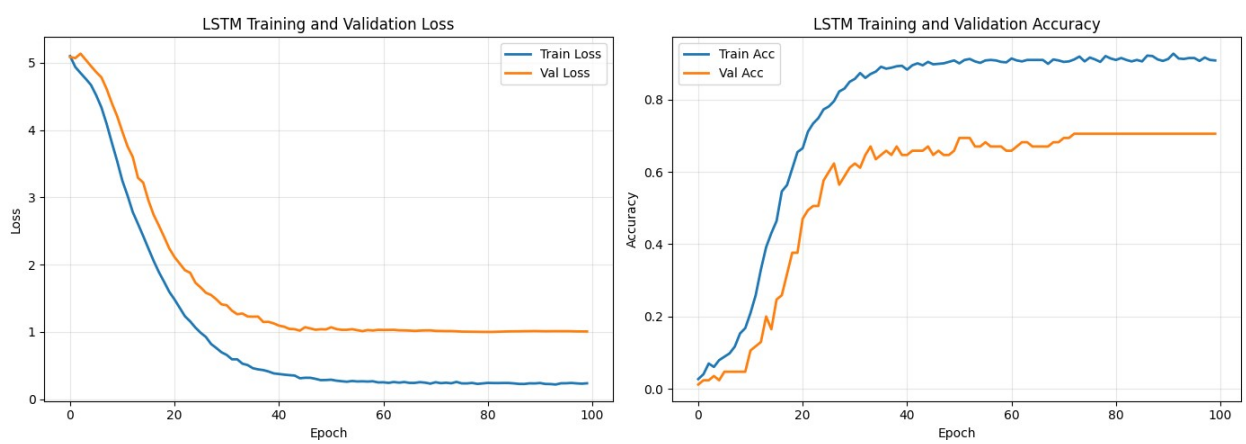
```
    return total_loss / len(dataloader), correct / total if total > 0
else 0

def validate(model, dataloader, criterion, device):
    model.eval()
    total_loss = 0
    correct = 0
    total = 0

    with torch.no_grad():
        for inputs, targets in dataloader:
            inputs, targets = inputs.to(device), targets.to(device)
            outputs, _ = model(inputs)

            loss = criterion(outputs.view(-1, model.vocab_size),
targets.view(-1))
            total_loss += loss.item()

            mask = targets != 0
            predictions = outputs.argmax(dim=-1)
            correct += ((predictions == targets) & mask).sum().item()
            total += mask.sum().item()

    return total_loss / len(dataloader), correct / total if total > 0
else 0

# Training loop
EPOCHS = 100
best_val_loss = float('inf')
training_history = {'train_loss': [], 'val_loss': [], 'train_acc': [],
'val_acc': []}

print("Starting LSTM training...")
start_time = time.time()

for epoch in range(EPOCHS):
    train_loss, train_acc = train_epoch(lstm_model, train_loader,
criterion, optimizer, device)
    val_loss, val_acc = validate(lstm_model, val_loader, criterion,
device)

    scheduler.step(val_loss)

    training_history['train_loss'].append(train_loss)
    training_history['val_loss'].append(val_loss)
    training_history['train_acc'].append(train_acc)
    training_history['val_acc'].append(val_acc)

    # Save best model
    if val_loss < best_val_loss:
```

```
        best_val_loss = val_loss
        torch.save({
            'epoch': epoch,
            'model_state_dict': lstm_model.state_dict(),
            'optimizer_state_dict': optimizer.state_dict(),
            'val_loss': val_loss,
            'vocab_size': vocab.n_words
        }, f'{RESULTS_DIR}/best_lstm_model.pt')

    if (epoch + 1) % 10 == 0:
        print(f"Epoch [{epoch+1}/{EPOCHS}] - "
              f"Train Loss: {train_loss:.4f}, Train Acc:
{train_acc:.4f} | "
              f"Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.4f}")

elapsed_time = time.time() - start_time
print(f"\nTraining completed in {elapsed_time/60:.2f} minutes")
print(f"Best validation loss: {best_val_loss:.4f}")
```

```
Starting LSTM training...
Epoch [10/100] - Train Loss: 3.5430, Train Acc: 0.1678 | Val Loss:
4.2100, Val Acc: 0.0471
Epoch [20/100] - Train Loss: 1.5962, Train Acc: 0.6550 | Val Loss:
2.2403, Val Acc: 0.3765
Epoch [30/100] - Train Loss: 0.6980, Train Acc: 0.8497 | Val Loss:
1.4102, Val Acc: 0.6118
Epoch [40/100] - Train Loss: 0.3845, Train Acc: 0.8940 | Val Loss:
1.1274, Val Acc: 0.6471
Epoch [50/100] - Train Loss: 0.2862, Train Acc: 0.9087 | Val Loss:
1.0387, Val Acc: 0.6588
Epoch [60/100] - Train Loss: 0.2503, Train Acc: 0.9034 | Val Loss:
1.0318, Val Acc: 0.6588
Epoch [70/100] - Train Loss: 0.2324, Train Acc: 0.9087 | Val Loss:
1.0236, Val Acc: 0.6824
Epoch [80/100] - Train Loss: 0.2354, Train Acc: 0.9141 | Val Loss:
1.0014, Val Acc: 0.7059
Epoch [90/100] - Train Loss: 0.2353, Train Acc: 0.9074 | Val Loss:
1.0136, Val Acc: 0.7059
Epoch [100/100] - Train Loss: 0.2375, Train Acc: 0.9087 | Val Loss:
1.0076, Val Acc: 0.7059

Training completed in 0.24 minutes
Best validation loss: 1.0009
```

```
# Cell 8: Plot Training History and Save
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Loss plot
axes[0].plot(training_history['train_loss'], label='Train Loss',
linewidth=2)
```

```
axes[0].plot(training_history['val_loss'], label='Val Loss',
linewidth=2)
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Loss')
axes[0].set_title('LSTM Training and Validation Loss')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Accuracy plot
axes[1].plot(training_history['train_acc'], label='Train Acc',
linewidth=2)
axes[1].plot(training_history['val_acc'], label='Val Acc',
linewidth=2)
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Accuracy')
axes[1].set_title('LSTM Training and Validation Accuracy')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(f'{RESULTS_DIR}/lstm_training_history.png', dpi=300,
bbox_inches='tight')
plt.show()

# Save training history
with open(f'{RESULTS_DIR}/lstm_training_history.json', 'w') as f:
    json.dump(training_history, f, indent=2)

print(f"Training plots and history saved to: {RESULTS_DIR}")
```




```
Training plots and history saved to:
/content/drive/MyDrive/BreakingBad_Sequence_Generation
```

```
# Cell 9: Text Generation Function for LSTM
def generate_text_lstm(model, vocab, seed_text: str, max_length: int =
20,
```

```
                        temperature: float = 0.8, device=device):
    model.eval()

    # Encode seed text
    words = seed_text.lower().split()
    input_indices = [vocab.word2idx.get(w, vocab.word2idx['<UNK>'])
for w in words]
    input_tensor = torch.tensor([input_indices],
dtype=torch.long).to(device)

    generated = words.copy()
    hidden = None

    with torch.no_grad():
        # Prime the model with seed
        for i in range(len(input_indices) - 1):
            _, hidden = model(input_tensor[:, i:i+1], hidden)

        current_input = input_tensor[:, -1:]

        # Generate new words
        for _ in range(max_length):
            output, hidden = model(current_input, hidden)

            # Apply temperature
            logits = output[0, -1] / temperature
            probs = torch.softmax(logits, dim=-1)

            # Sample from distribution
            next_token = torch.multinomial(probs, 1).item()

            # Stop if EOS
            if next_token == vocab.word2idx['<EOS>']:
                break

            word = vocab.idx2word.get(next_token, '<UNK>')
            generated.append(word)

            current_input = torch.tensor([[next_token]],
dtype=torch.long).to(device)

    return ' '.join(generated)

# Load best model for generation
checkpoint = torch.load(f'{RESULTS_DIR}/best_lstm_model.pt')
lstm_model.load_state_dict(checkpoint['model_state_dict'])
lstm_model.eval()

# Generate samples with different seeds
seed_phrases = [
```

```
    "walter white",
    "jesse pinkman",
    "chemistry is",
    "the empire",
    "breaking bad"
]

lstm_results = []
print("=" * 60)
print("LSTM GENERATED SAMPLES")
print("=" * 60)

for seed in seed_phrases:
    print(f"\nSeed: '{seed}'")
    print("-" * 40)
    for temp in [0.5, 0.8, 1.0]:
        generated = generate_text_lstm(lstm_model, vocab, seed,
                                       max_length=15,
temperature=temp)
        print(f"Temp {temp}: {generated}")
        lstm_results.append({
            'model': 'LSTM',
            'seed': seed,
            'temperature': temp,
            'generated_text': generated
        })
    print()

# Save generation results
df_lstm = pd.DataFrame(lstm_results)
df_lstm.to_excel(f'{RESULTS_DIR}/lstm_generation_results.xlsx',
index=False)
print(f"Results saved to: {RESULTS_DIR}/lstm_generation_results.xlsx")
```

```
============================================================
LSTM GENERATED SAMPLES
============================================================

Seed: 'walter white'
----------------------------------------
Temp 0.5: walter white believes chemistry is at its core
Temp 0.8: walter white believes chemistry is transformation at its
core
Temp 1.0: walter white believes chemistry is is matter


Seed: 'jesse pinkman'
----------------------------------------
Temp 0.5: jesse pinkman learned that consequences follow every
decision
```

```
Temp 0.8: jesse pinkman learned that consequences follow every
decision
Temp 1.0: jesse pinkman learned that consequences follow every
decision


Seed: 'chemistry is'
----------------------------------------
Temp 0.5: chemistry is transformation at its core
Temp 0.8: chemistry is transformation at its core
Temp 1.0: chemistry is at at its core


Seed: 'the empire'
----------------------------------------
Temp 0.5: the empire never releases its members
Temp 0.8: the empire never releases its members
Temp 1.0: the empire never releases its members


Seed: 'breaking bad'
----------------------------------------
Temp 0.5: breaking bad means choosing a darker path over the right one
Temp 0.8: breaking bad means choosing a darker path over the right one
Temp 1.0: breaking bad means choosing a darker path over the right one

Results saved to:
/content/drive/MyDrive/BreakingBad_Sequence_Generation/lstm_generation
_results.xlsx
```

```python
# Cell 10: Define Transformer-based Generative Model
class PositionalEncoding(nn.Module):
    def __init__(self, d_model: int, max_len: int = 5000, dropout:
float = 0.1):
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)

        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len,
dtype=torch.float).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() *
                             (-np.log(10000.0) / d_model))

        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        pe = pe.unsqueeze(0).transpose(0, 1)

        self.register_buffer('pe', pe)

    def forward(self, x):
        x = x + self.pe[:x.size(0), :]
```

```
        return self.dropout(x)

class TransformerGenerator(nn.Module):
    def __init__(self, vocab_size: int, d_model: int = 128, nhead: int
= 4,
                 num_layers: int = 2, dim_feedforward: int = 512,
dropout: float = 0.3):
        super(TransformerGenerator, self).__init__()

        self.d_model = d_model
        self.vocab_size = vocab_size

        # Embedding
        self.embedding = nn.Embedding(vocab_size, d_model)
        self.pos_encoder = PositionalEncoding(d_model,
dropout=dropout)

        # Transformer
        encoder_layers = nn.TransformerEncoderLayer(
            d_model=d_model, nhead=nhead,
dim_feedforward=dim_feedforward,
            dropout=dropout, batch_first=True
        )
        self.transformer_encoder =
nn.TransformerEncoder(encoder_layers, num_layers)

        # Output
        self.decoder = nn.Linear(d_model, vocab_size)
        self.dropout = nn.Dropout(dropout)

        self.init_weights()

    def init_weights(self):
        initrange = 0.1
        self.embedding.weight.data.uniform_(-initrange, initrange)
        self.decoder.bias.data.zero_()
        self.decoder.weight.data.uniform_(-initrange, initrange)

    def forward(self, src, src_mask=None):
        # src shape: (batch, seq_len)
        src = self.embedding(src) * np.sqrt(self.d_model)
        src = self.pos_encoder(src)

        if src_mask is None:
            src_mask =
self.generate_square_subsequent_mask(src.size(1)).to(src.device)

        output = self.transformer_encoder(src, mask=src_mask)
        output = self.dropout(output)
        output = self.decoder(output)
```

```
        return output

    def generate_square_subsequent_mask(self, sz):
        mask = torch.triu(torch.ones(sz, sz), diagonal=1)
        mask = mask.masked_fill(mask == 1, float('-inf'))
        return mask

# Initialize Transformer model
D_MODEL = 128
NHEAD = 4
NUM_TRANSFORMER_LAYERS = 2

transformer_model = TransformerGenerator(
    vocab.n_words, D_MODEL, NHEAD, NUM_TRANSFORMER_LAYERS
).to(device)

print(f"Transformer parameters: {sum(p.numel() for p in
transformer_model.parameters()):,}")

# Optimizer and scheduler
optimizer_trans = optim.AdamW(transformer_model.parameters(),
lr=0.001, weight_decay=1e-5)
scheduler_trans =
optim.lr_scheduler.ReduceLROnPlateau(optimizer_trans, mode='min',
factor=0.5, patience=5)
```

```
Transformer parameters: 439,977
```

```
# Cell 11: Training Function for Transformer
def train_epoch_transformer(model, dataloader, criterion, optimizer,
device):
    model.train()
    total_loss = 0
    correct = 0
    total = 0

    for inputs, targets in dataloader:
        inputs, targets = inputs.to(device), targets.to(device)

        optimizer.zero_grad()
        outputs = model(inputs)

        loss = criterion(outputs.view(-1, model.vocab_size),
targets.view(-1))
        loss.backward()

        torch.nn.utils.clip_grad_norm_(model.parameters(),
max_norm=1.0)
        optimizer.step()
```

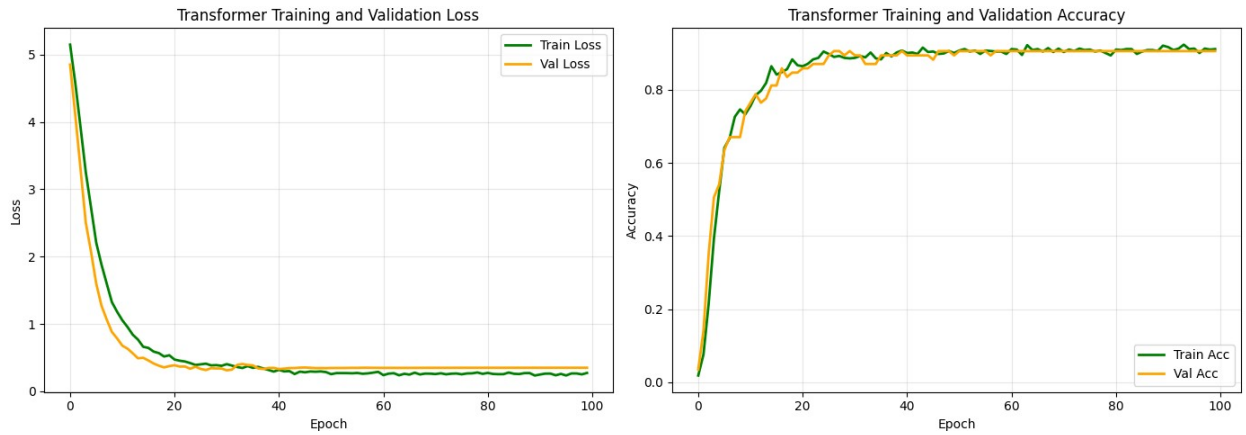
```
        total_loss += loss.item()

        # Accuracy
        mask = targets != 0
        predictions = outputs.argmax(dim=-1)
        correct += ((predictions == targets) & mask).sum().item()
        total += mask.sum().item()

    return total_loss / len(dataloader), correct / total if total > 0
else 0

def validate_transformer(model, dataloader, criterion, device):
    model.eval()
    total_loss = 0
    correct = 0
    total = 0

    with torch.no_grad():
        for inputs, targets in dataloader:
            inputs, targets = inputs.to(device), targets.to(device)
            outputs = model(inputs)

            loss = criterion(outputs.view(-1, model.vocab_size),
targets.view(-1))
            total_loss += loss.item()

            mask = targets != 0
            predictions = outputs.argmax(dim=-1)
            correct += ((predictions == targets) & mask).sum().item()
            total += mask.sum().item()

    return total_loss / len(dataloader), correct / total if total > 0
else 0

# Training loop for Transformer
EPOCHS_TRANS = 100
best_val_loss_trans = float('inf')
trans_history = {'train_loss': [], 'val_loss': [], 'train_acc': [],
'val_acc': []}

print("Starting Transformer training...")
start_time = time.time()

for epoch in range(EPOCHS_TRANS):
    train_loss, train_acc = train_epoch_transformer(
        transformer_model, train_loader, criterion, optimizer_trans,
device
    )
    val_loss, val_acc = validate_transformer(
        transformer_model, val_loader, criterion, device
```

```
    )

    scheduler_trans.step(val_loss)

    trans_history['train_loss'].append(train_loss)
    trans_history['val_loss'].append(val_loss)
    trans_history['train_acc'].append(train_acc)
    trans_history['val_acc'].append(val_acc)

    if val_loss < best_val_loss_trans:
        best_val_loss_trans = val_loss
        torch.save({
            'epoch': epoch,
            'model_state_dict': transformer_model.state_dict(),
            'optimizer_state_dict': optimizer_trans.state_dict(),
            'val_loss': val_loss,
        }, f'{RESULTS_DIR}/best_transformer_model.pt')

    if (epoch + 1) % 10 == 0:
        print(f"Epoch [{epoch+1}/{EPOCHS_TRANS}] - "
              f"Train Loss: {train_loss:.4f}, Train Acc:
{train_acc:.4f} | "
              f"Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.4f}")

elapsed_time = time.time() - start_time
print(f"\nTransformer training completed in {elapsed_time/60:.2f}
minutes")
print(f"Best validation loss: {best_val_loss_trans:.4f}")
```

```
Starting Transformer training...
Epoch [10/100] - Train Loss: 1.1810, Train Acc: 0.7329 | Val Loss:
0.7853, Val Acc: 0.7412
Epoch [20/100] - Train Loss: 0.5349, Train Acc: 0.8671 | Val Loss:
0.3737, Val Acc: 0.8471
Epoch [30/100] - Train Loss: 0.3781, Train Acc: 0.8859 | Val Loss:
0.3381, Val Acc: 0.9059
Epoch [40/100] - Train Loss: 0.2925, Train Acc: 0.9074 | Val Loss:
0.3477, Val Acc: 0.9059
Epoch [50/100] - Train Loss: 0.2851, Train Acc: 0.9007 | Val Loss:
0.3431, Val Acc: 0.8941
Epoch [60/100] - Train Loss: 0.2886, Train Acc: 0.8980 | Val Loss:
0.3482, Val Acc: 0.9059
Epoch [70/100] - Train Loss: 0.2618, Train Acc: 0.9128 | Val Loss:
0.3487, Val Acc: 0.9059
Epoch [80/100] - Train Loss: 0.2592, Train Acc: 0.8940 | Val Loss:
0.3491, Val Acc: 0.9059
Epoch [90/100] - Train Loss: 0.2342, Train Acc: 0.9208 | Val Loss:
0.3495, Val Acc: 0.9059
Epoch [100/100] - Train Loss: 0.2723, Train Acc: 0.9114 | Val Loss:
0.3496, Val Acc: 0.9059
```

```
Transformer training completed in 0.28 minutes
Best validation loss: 0.3115
```

```python
# Cell 12: Plot Transformer Training History
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(trans_history['train_loss'], label='Train Loss',
linewidth=2, color='green')
axes[0].plot(trans_history['val_loss'], label='Val Loss', linewidth=2,
color='orange')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Loss')
axes[0].set_title('Transformer Training and Validation Loss')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].plot(trans_history['train_acc'], label='Train Acc',
linewidth=2, color='green')
axes[1].plot(trans_history['val_acc'], label='Val Acc', linewidth=2,
color='orange')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Accuracy')
axes[1].set_title('Transformer Training and Validation Accuracy')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(f'{RESULTS_DIR}/transformer_training_history.png',
dpi=300, bbox_inches='tight')
plt.show()

with open(f'{RESULTS_DIR}/transformer_training_history.json', 'w') as
f:
    json.dump(trans_history, f, indent=2)

print(f"Transformer training plots saved to: {RESULTS_DIR}")
```




```
Transformer training plots saved to:
/content/drive/MyDrive/BreakingBad_Sequence_Generation

# Cell 13: Text Generation Function for Transformer
def generate_text_transformer(model, vocab, seed_text: str,
max_length: int = 20,
                              temperature: float = 0.8,
device=device):
    model.eval()

    words = seed_text.lower().split()
    input_indices = [vocab.word2idx.get(w, vocab.word2idx['<UNK>'])
for w in words]

    generated = words.copy()

    with torch.no_grad():
        for _ in range(max_length):
            input_tensor = torch.tensor([input_indices],
dtype=torch.long).to(device)
            output = model(input_tensor)

            logits = output[0, -1] / temperature
            probs = torch.softmax(logits, dim=-1)

            next_token = torch.multinomial(probs, 1).item()

            if next_token == vocab.word2idx['<EOS>']:
                break

            word = vocab.idx2word.get(next_token, '<UNK>')
            generated.append(word)
            input_indices.append(next_token)

            # Keep window size manageable
            if len(input_indices) > SEQ_LENGTH:
                input_indices = input_indices[-SEQ_LENGTH:]
```

```
    return ' '.join(generated)

# Load best transformer model
checkpoint_trans =
torch.load(f'{RESULTS_DIR}/best_transformer_model.pt')
transformer_model.load_state_dict(checkpoint_trans['model_state_dict']
)
transformer_model.eval()

# Generate samples
transformer_results = []
print("=" * 60)
print("TRANSFORMER GENERATED SAMPLES")
print("=" * 60)

for seed in seed_phrases:
    print(f"\nSeed: '{seed}'")
    print("-" * 40)
    for temp in [0.5, 0.8, 1.0]:
        generated = generate_text_transformer(
            transformer_model, vocab, seed, max_length=15,
temperature=temp
        )
        print(f"Temp {temp}: {generated}")
        transformer_results.append({
            'model': 'Transformer',
            'seed': seed,
            'temperature': temp,
            'generated_text': generated
        })
    print()

# Save results
df_transformer = pd.DataFrame(transformer_results)
df_transformer.to_excel(f'{RESULTS_DIR}/transformer_generation_results
.xlsx', index=False)
print(f"Results saved to:
{RESULTS_DIR}/transformer_generation_results.xlsx")
```

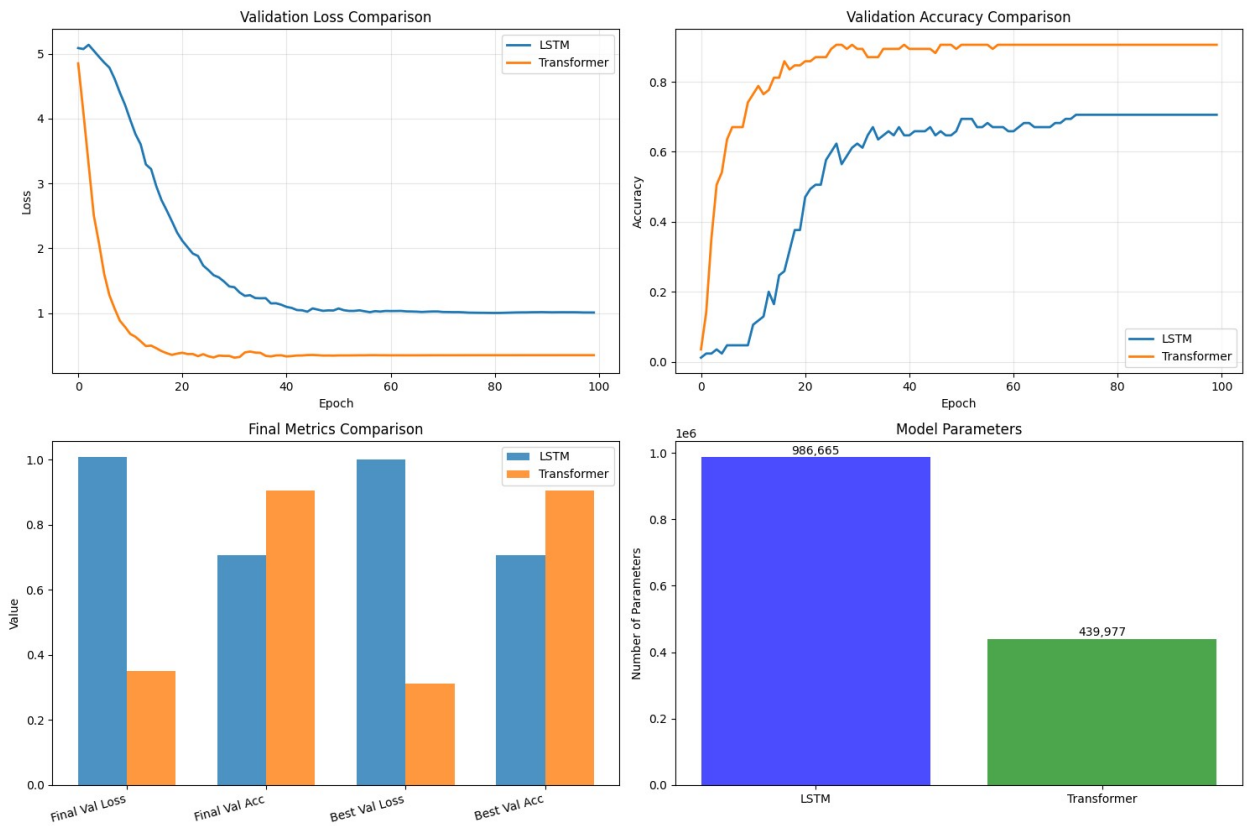
```
============================================================
TRANSFORMER GENERATED SAMPLES
============================================================

Seed: 'walter white'
----------------------------------------
Temp 0.5: walter white believes chemistry is transformation at its
core
Temp 0.8: walter white believes chemistry is transformation at its
members
```

```
Temp 1.0: walter white builds a criminal network step by sequence by
sequence by sequence by sequence by sequence


Seed: 'jesse pinkman'
----------------------------------------
Temp 0.5: jesse pinkman learned that consequences follow every
decision
Temp 0.8: jesse pinkman learned that consequences follow every
decision
Temp 1.0: jesse pinkman learned that consequences follow every
decision


Seed: 'chemistry is'
----------------------------------------
Temp 0.5: chemistry is transformation at its core
Temp 0.8: chemistry is transformation at its core
Temp 1.0: chemistry is transformation at its core


Seed: 'the empire'
----------------------------------------
Temp 0.5: the empire never releases its members
Temp 0.8: the empire never releases its members
Temp 1.0: the empire never releases its members


Seed: 'breaking bad'
----------------------------------------
Temp 0.5: breaking bad means choosing a darker path over the right one
Temp 0.8: breaking bad means choosing a darker path over the right one
Temp 1.0: breaking bad means choosing a darker path over the right one

Results saved to:
/content/drive/MyDrive/BreakingBad_Sequence_Generation/transformer_gen
eration_results.xlsx
```

```python
# Cell 14: Compare Both Models and Create Summary
# Combine all results
all_results = lstm_results + transformer_results
df_all = pd.DataFrame(all_results)
df_all.to_excel(f'{RESULTS_DIR}/all_generation_results.xlsx',
index=False)

# Create comparison visualization
fig, axes = plt.subplots(2, 2, figsize=(15, 10))

# Loss comparison
axes[0, 0].plot(training_history['val_loss'], label='LSTM',
linewidth=2)
```

```
axes[0, 0].plot(trans_history['val_loss'], label='Transformer',
linewidth=2)
axes[0, 0].set_title('Validation Loss Comparison')
axes[0, 0].set_xlabel('Epoch')
axes[0, 0].set_ylabel('Loss')
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

# Accuracy comparison
axes[0, 1].plot(training_history['val_acc'], label='LSTM',
linewidth=2)
axes[0, 1].plot(trans_history['val_acc'], label='Transformer',
linewidth=2)
axes[0, 1].set_title('Validation Accuracy Comparison')
axes[0, 1].set_xlabel('Epoch')
axes[0, 1].set_ylabel('Accuracy')
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# Final metrics comparison
metrics = ['Final Val Loss', 'Final Val Acc', 'Best Val Loss', 'Best
Val Acc']
lstm_vals = [
    training_history['val_loss'][-1],
    training_history['val_acc'][-1],
    min(training_history['val_loss']),
    max(training_history['val_acc'])
]
trans_vals = [
    trans_history['val_loss'][-1],
    trans_history['val_acc'][-1],
    min(trans_history['val_loss']),
    max(trans_history['val_acc'])
]

x = np.arange(len(metrics))
width = 0.35
axes[1, 0].bar(x - width/2, lstm_vals, width, label='LSTM', alpha=0.8)
axes[1, 0].bar(x + width/2, trans_vals, width, label='Transformer',
alpha=0.8)
axes[1, 0].set_ylabel('Value')
axes[1, 0].set_title('Final Metrics Comparison')
axes[1, 0].set_xticks(x)
axes[1, 0].set_xticklabels(metrics, rotation=15, ha='right')
axes[1, 0].legend()

# Model complexity
model_names = ['LSTM', 'Transformer']
param_counts = [
    sum(p.numel() for p in lstm_model.parameters()),
```

```
    sum(p.numel() for p in transformer_model.parameters())
]
axes[1, 1].bar(model_names, param_counts, color=['blue', 'green'],
alpha=0.7)
axes[1, 1].set_title('Model Parameters')
axes[1, 1].set_ylabel('Number of Parameters')

for i, v in enumerate(param_counts):
    axes[1, 1].text(i, v + 1000, f'{v:,}', ha='center', va='bottom')

plt.tight_layout()
plt.savefig(f'{RESULTS_DIR}/model_comparison.png', dpi=300,
bbox_inches='tight')
plt.show()

print(f"Comparison plots saved to:
{RESULTS_DIR}/model_comparison.png")
```




```
Comparison plots saved to:
/content/drive/MyDrive/BreakingBad_Sequence_Generation/model_compariso
n.png

# Cell 15: Generate Final Report and Save All Artifacts
report = f"""
```

```
BREAKING BAD SEQUENCE GENERATION LAB - FINAL REPORT
{'='*70}

Dataset: Breaking Bad themed sentences (25 sentences)
Vocabulary Size: {vocab.n_words}
Sequence Length: {SEQ_LENGTH}
Training Samples: {len(train_dataset)}
Validation Samples: {len(val_dataset)}

{'='*70}
LSTM MODEL RESULTS
{'='*70}
Embedding Dimension: {EMBED_DIM}
Hidden Dimension: {HIDDEN_DIM}
Number of Layers: {NUM_LAYERS}
Total Parameters: {sum(p.numel() for p in lstm_model.parameters()):,}
Best Validation Loss: {min(training_history['val_loss']):.4f}
Best Validation Accuracy: {max(training_history['val_acc']):.4f}
Final Validation Loss: {training_history['val_loss'][-1]:.4f}
Final Validation Accuracy: {training_history['val_acc'][-1]:.4f}

{'='*70}
TRANSFORMER MODEL RESULTS
{'='*70}
Model Dimension: {D_MODEL}
Number of Heads: {NHEAD}
Number of Layers: {NUM_TRANSFORMER_LAYERS}
Total Parameters: {sum(p.numel() for p in
transformer_model.parameters()):,}
Best Validation Loss: {min(trans_history['val_loss']):.4f}
Best Validation Accuracy: {max(trans_history['val_acc']):.4f}
Final Validation Loss: {trans_history['val_loss'][-1]:.4f}
Final Validation Accuracy: {trans_history['val_acc'][-1]:.4f}

{'='*70}
SAVED FILES
{'='*70}
- vocabulary.pkl: Saved vocabulary object
- best_lstm_model.pt: Best LSTM model checkpoint
- best_transformer_model.pt: Best Transformer model checkpoint
- lstm_training_history.png: LSTM training curves
- transformer_training_history.png: Transformer training curves
- model_comparison.png: Side-by-side model comparison
- lstm_generation_results.xlsx: LSTM text generation samples
- transformer_generation_results.xlsx: Transformer text generation
samples
- all_generation_results.xlsx: Combined generation results
- lstm_training_history.json: Raw LSTM training data
- transformer_training_history.json: Raw Transformer training data
```

```
Generated on: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}
"""

print(report)

# Save report
with open(f'{RESULTS_DIR}/final_report.txt', 'w') as f:
    f.write(report)

print(f"\nAll results saved to Google Drive: {RESULTS_DIR}")
```

```
BREAKING BAD SEQUENCE GENERATION LAB - FINAL REPORT
======================================================================

Dataset: Breaking Bad themed sentences (25 sentences)
Vocabulary Size: 169
Sequence Length: 5
Training Samples: 149
Validation Samples: 17

======================================================================
LSTM MODEL RESULTS
======================================================================
Embedding Dimension: 128
Hidden Dimension: 256
Number of Layers: 2
Total Parameters: 986,665
Best Validation Loss: 1.0009
Best Validation Accuracy: 0.7059
Final Validation Loss: 1.0076
Final Validation Accuracy: 0.7059

======================================================================
TRANSFORMER MODEL RESULTS
======================================================================
Model Dimension: 128
Number of Heads: 4
Number of Layers: 2
Total Parameters: 439,977
Best Validation Loss: 0.3115
Best Validation Accuracy: 0.9059
Final Validation Loss: 0.3496
Final Validation Accuracy: 0.9059

======================================================================
SAVED FILES
======================================================================
- vocabulary.pkl: Saved vocabulary object
- best_lstm_model.pt: Best LSTM model checkpoint
```

```
- best_transformer_model.pt: Best Transformer model checkpoint
- lstm_training_history.png: LSTM training curves
- transformer_training_history.png: Transformer training curves
- model_comparison.png: Side-by-side model comparison
- lstm_generation_results.xlsx: LSTM text generation samples
- transformer_generation_results.xlsx: Transformer text generation
samples
- all_generation_results.xlsx: Combined generation results
- lstm_training_history.json: Raw LSTM training data
- transformer_training_history.json: Raw Transformer training data

Generated on: 2026-03-20 10:28:01


All results saved to Google Drive:
/content/drive/MyDrive/BreakingBad_Sequence_Generation
```

```python
# Cell 16: Interactive Generation - Try Your Own Seeds!
def interactive_generate(seed_text, model_type='both',
temperature=0.8, max_length=15):
    print(f"\nSeed: '{seed_text}'")
    print("-" * 50)

    if model_type in ['lstm', 'both']:
        result = generate_text_lstm(lstm_model, vocab, seed_text,
max_length, temperature)
        print(f"LSTM:      {result}")

    if model_type in ['transformer', 'both']:
        result = generate_text_transformer(transformer_model, vocab,
seed_text, max_length, temperature)
        print(f"Transformer: {result}")

# Test with custom seeds
test_seeds = [
    "walter and jesse",
    "the blue meth",
    "say my name",
    "i am the danger"
]

print("=" * 60)
print("CUSTOM GENERATION EXAMPLES")
print("=" * 60)

for seed in test_seeds:
    interactive_generate(seed, model_type='both', temperature=0.7)
```

```
============================================================
CUSTOM GENERATION EXAMPLES
```

```
============================================================

Seed: 'walter and jesse'
--------------------------------------------------
LSTM:      walter and jesse own no matter the cost
Transformer: walter and jesse that emotions are a weakness in business

Seed: 'the blue meth'
--------------------------------------------------
LSTM:      the blue meth becomes a symbol of perfection and obsession
Transformer: the blue meth

Seed: 'say my name'
--------------------------------------------------
LSTM:      say my name and discipline
Transformer: say my name

Seed: 'i am the danger'
--------------------------------------------------
LSTM:      i am the danger
Transformer: i am the danger real
```